

Inference in the log domain

Simon Frost

Table of contents

1	Overview	1
2	Setup	2
3	A model whose evidence underflows	2
4	Where ordinary arithmetic gives out	3
5	Evidence mass, and telling rare from impossible	4
6	Every marginal at once	5
7	What this does not promise	5
8	Summary	6
9	References	6

1 Overview

Every backend of the previous vignettes multiplies probabilities together. On a small network that is harmless, but the joint probability of a long run of observations is a product of many numbers below one, and Float64 runs out of exponent at about 10^{-308} — and out of *precision* well before that. When the evidence mass underflows to zero, an exact backend cannot form a posterior at all: dividing by zero mass is not a rounding error, it is a missing answer.

The awkward part is that a genuinely impossible observation also has zero mass. In ordinary arithmetic the two are indistinguishable, and the error is the same.

This vignette covers the opt-in log-domain layer, which addresses both:

- `LogVariableElimination` and `LogJunctionTree` carry centered log-domain factors and combine them with log-sum-exp, so the intermediate quantities stay representable.

- `log_evidence_probability` returns the log evidence mass, which is an ordinary finite number for evidence that is merely rare, and `-Inf` for evidence that is impossible.
- `LogInferenceDiagnostics.mass_status` reports `:finite`, `:underflow` or `:zero`, so a run says which situation it was in.

The default backends are unchanged. This layer is opt-in, and the last section is explicit about what it does *not* promise.

2 Setup

```
using BayesianNetworkInference
using FiniteKernels
using BayesianNetworks
```

3 A model whose evidence underflows

A chain of repeated detections is the simplest way to get a very small but perfectly ordinary evidence probability. D1 is rarely positive, and each subsequent site copies the previous one with a little noise:

```
function detection_chain(n)
    vars = [Symbol("D", i) for i in 1:n]
    bn = bayesnet([v => [:no, :yes] for v in vars]...;
                  mechanisms = vcat([vars[1] => ()],
                                    [vars[i] => (vars[i - 1],) for i in 2:n]))
    m = bind_cpt(BayesModel(bn), vars[1] => [0.999, 0.001])
    for i in 2:n
        m = bind_cpt(m, vars[i] => [0.999 0.001; 0.9 0.1])
    end
    return m, vars
end

m, vars = detection_chain(5)
length(vars)
```

5

Observing `:yes` at every site but the last is legitimate evidence: it just becomes less and less likely as the chain grows. The chain is Markov, so the answer is known in closed form and does not depend on the length at all — conditioning on the previous site leaves the last one depending on nothing else, so the posterior is the second row of the transition table:

```
exact = [0.9, 0.1]
```

```
2-element Vector{Float64}:  
 0.9  
 0.1
```

That closed form is the oracle for everything below.

4 Where ordinary arithmetic gives out

Running the default backend as the chain grows shows three regimes rather than two.

```
function compare(n)  
    mm, vv = detection_chain(n)  
    ev = Dict{v => :yes for v in vv[1:(n - 1)]}  
    default = try  
        p, _ = infer(mm, vv[n]; evidence = ev)  
        round.(p.table; digits = 4)  
    catch e  
        typeof(e)  
    end  
    p, d = infer(mm, vv[n]; evidence = ev, backend = LogVariableElimination())  
    return (n = n, default = default, log_domain = round.(p.table; digits = 4),  
            log_mass = round(d.log_evidence_probability; digits = 1),  
            status = d.mass_status)  
end  
  
[compare(n) for n in (300, 320, 340, 400)]
```

```
4-element Vector{NamedTuple{(:n, :default, :log_domain, :log_mass, :status)}}:  
 (n = 300, default = [0.9, 0.1], log_domain = [0.9, 0.1], log_mass = -693.1, status  
 = :finite)  
 (n = 320, default = [0.901, 0.099], log_domain = [0.9, 0.1], log_mass = -739.1,  
 status = :finite)  
 (n = 340, default = KernelNormalizationError, log_domain = [0.9, 0.1], log_mass =  
 -785.2, status = :underflow)  
 (n = 400, default = KernelNormalizationError, log_domain = [0.9, 0.1], log_mass =  
 -923.3, status = :underflow)
```

At 300 sites the two agree. At 320 the default backend still returns an answer, but it is no longer the right one — the fourth decimal has moved, because the mass is near the bottom of the representable range and the significand has been eaten away. At 340 the mass underflows to zero and the default backend raises, correctly, because it cannot form a posterior from zero.

The log-domain backend returns the exact answer throughout:

```
all(compare(n).log_domain == exact for n in (300, 320, 340, 400))
```

true

The middle regime is the one worth dwelling on: silent precision loss arrives *before* outright failure, and only the closed form reveals it.

5 Evidence mass, and telling rare from impossible

`log_evidence_probability` reports the mass directly, without forming a posterior:

```
m340, v340 = detection_chain(340)
rare = Dict(v ⇒ :yes for v in v340[1:339])
log_evidence_probability(m340; evidence = rare)
```

-785.181516710967

A finite number, far below `floatmin(Float64)` but entirely well defined. Contrast it with evidence that genuinely cannot happen, here forced by a deterministic mechanism:

```
det = bayesnet(:A ⇒ [:no, :yes], :B ⇒ [:no, :yes], :C ⇒ [:lo, :hi];
              mechanisms = [:A ⇒ (), :B ⇒ (:A,), :C ⇒ (:A,)])
dm = bind_cpt(BayesModel(det),
             [:A ⇒ [0.5, 0.5], :B ⇒ [1.0 0.0; 0.0 1.0], :C ⇒ [0.5 0.5; 0.2
0.8]])
impossible = Dict(:A ⇒ :yes, :B ⇒ :no)
log_evidence_probability(dm; evidence = impossible)
```

-Inf

-Inf, not a small number. The two cases are now distinguishable, which they are not in ordinary arithmetic: there, both the rare chain and the contradictory observation produce zero mass and the same error.

```
(rare_chain = try
  infer(m340, :D340; evidence = rare)
catch e
  typeof(e)
end,
contradiction = try
  infer(dm, :C; evidence = impossible)
catch e
  typeof(e)
end)
```

(rare_chain = KernelNormalizationError, contradiction = KernelNormalizationError)

Both raise, and the two situations are not the same: one answer exists and cannot be reached in Float64, the other does not exist. `mass_status` records which:

```
(rare = compare(340).status, ordinary = compare(20).status)
```

```
(rare = :underflow, ordinary = :finite)
```

An impossible observation still raises under the log backend — there is no posterior to return — but `log_evidence_probability` gives `-Inf` rather than an exception, so a caller can test feasibility before committing to a query.

6 Every marginal at once

`LogJunctionTree` applies the same arithmetic to the Shafer-Shenoy collect/distribute schedule of the previous vignette ([Shafer and Shenoy 1990](#)), so one calibration answers every variable:

```
posteriors = all_marginals(m340; evidence = rare, backend = LogJunctionTree())  
length(posteriors), round.(posteriors[:D340].table; digits = 4)
```

```
(340, [0.9, 0.1])
```

`log_calibrate` exposes the calibrated tree itself, with the log evidence and the centered component beliefs, for callers that want the intermediate quantities rather than the marginals:

```
calibrated = log_calibrate(compile(m340); evidence = rare)  
typeof(calibrated)
```

`LogCalibratedJunctionTree`

A query spanning several cliques falls back to log variable elimination and says so in its diagnostics, exactly as the ordinary junction tree does.

7 What this does not promise

The backend returns Float64 posterior *cells*. Keeping the intermediate mass in the log domain prevents the intermediate product from underflowing; it does not make an unrepresentably small answer representable. A posterior cell below `floatmin` still underflows on the way out, and this is not a general error bound:

```
floatmin(Float64), exp(-785.2)
```

```
(2.2250738585072014e-308, 0.0)
```

Two further limits, both deliberate. Factor entries must be nonnegative and finite — a negative entry has no logarithm, and the layer raises rather than silently producing NaN:

```
try
    infer(FactorGraph([Factor(FiniteAxis(:X, [:a, :b]), [0.5, -0.5]))], [:X];
        backend = LogVariableElimination())
catch e
    typeof(e)
end
```

LogFactorDomainError

And an empty query keeps the ordinary unnormalised-mass API, with the log mass available in the diagnostics rather than replacing the returned value.

8 Summary

Log-domain inference is not a different algorithm: `LogVariableElimination` and `LogJunctionTree` run the same elimination order and the same message schedule as their ordinary counterparts (Dechter 1999; Koller and Friedman 2009), with products replaced by sums and sums by log-sum-exp on centered factors. What it buys is the range to carry evidence that is merely rare, and — because a finite log mass and `-Inf` are different numbers, whereas zero and zero are not — the ability to say whether an observation was too small to represent or genuinely impossible. What it does not buy is a bound on the error of a posterior cell, which is why the default backends are unchanged and this layer is opt-in.

The exact-arithmetic decision path of `InfluenceDiagrams.jl` answers the same concern for decisions rather than posteriors, where the quantity at risk is a sum of signed utilities rather than a product of probabilities.

9 References

- Dechter, Rina. 1999. “Bucket Elimination: A Unifying Framework for Reasoning.” *Artificial Intelligence* 113 (1–2): 41–85. [https://doi.org/10.1016/S0004-3702\(99\)00059-4](https://doi.org/10.1016/S0004-3702(99)00059-4).
- Koller, Daphne, and Nir Friedman. 2009. *Probabilistic Graphical Models: Principles and Techniques*. MIT Press.
- Shafer, Glenn R., and Prakash P. Shenoy. 1990. “Probability Propagation.” *Annals of Mathematics and Artificial Intelligence* 2: 327–51. <https://doi.org/10.1007/BF01531015>.